

Structure de la base de données de la plateforme de gestion commerciale

De l'idée métier à l'implémentation PostgreSQL réelle : règles d'élaboration d'un système d'information, dictionnaire de données, modèle conceptuel, modèle logique, modèle relationnel et création physique — tables, contraintes, index, triggers et séquences.

Méthode Merise (MCD → MLD → MPD)

PostgreSQL 15+

30 tables · 6 domaines fonctionnels

Sommaire

- 0 **Présentation du projet** Idée principale, problématique métier, objectifs, périmètre fonctionnel

- 1 **Règles d'élaboration d'un système d'information** Démarche méthodologique Merise, règles de gestion réelles du projet

- 2 **Dictionnaire de données, associations et cardinalités** Recensement complet des propriétés, des liens entre elles et de leur cardinalité

- 3 **Règles de normalisation appliquées** 1FN, 2FN, 3FN — comment le dictionnaire est nettoyé et affiné avant modélisation

- 4 **Construction du Modèle Conceptuel de Données (MCD)** 6 sous-domaines fonctionnels, entités, associations, cardinalités — schémas

- 5 **Passage au Modèle Logique de Données (MLD)** Règles de transformation MCD → MLD appliquées à chaque domaine

- 6 **Passage au Modèle Relationnel** Formalisation en relations, clés primaires et étrangères

- 7 **Création de la base PostgreSQL** Tables, contraintes, index, triggers, séquences — implémentation réelle et concepts PostgreSQL

- 8 **Synthèse** Bilan chiffré et vue d'ensemble du schéma final

NOTE DE MÉTHODE

Ce document décrit la base de données **réellement construite et en service** pour cette plateforme. Il est rédigé comme une soutenance : chaque étape de la méthode Merise est appliquée dans l'ordre académique classique, puis systématiquement reliée à la table, la contrainte ou le mécanisme PostgreSQL exact qui l'implémente dans ce projet précis — jamais un exemple générique.

Présentation du projet

0.1 Idée principale

La plateforme est un **logiciel de gestion commerciale en mode SaaS (Software as a Service), multi-boutiques et multi-utilisateurs**, destiné aux petits commerces guinéens (téléphonie, alimentation générale, vêtements, quincaillerie, pharmacie, électronique, cosmétique...). Chaque boutique cliente dispose de son propre espace strictement cloisonné (catalogue, stock, ventes, clients, équipe), hébergé sur une infrastructure et une base de données **partagées** entre tous les clients de la plateforme — c'est le principe du **multi-tenant** : une seule application, une seule base, mais des données rigoureusement séparées boutique par boutique.

Le cœur du système est le **point de vente (caisse)** : un vendeur encaisse une vente, le stock se met à jour automatiquement, et toute l'activité (chiffre d'affaires, bénéfice, produits les plus vendus) devient immédiatement visible au propriétaire. Autour de ce cœur gravitent la gestion du catalogue produit, les clients et leurs créances, les fournisseurs et les commandes d'achat, la supervision à distance d'une boutique par son propriétaire, et la facturation de l'abonnement à la plateforme elle-même.

0.2 Problématique métier

Dans le contexte guinéen visé, la majorité des petits commerces gèrent aujourd'hui leur activité de façon informelle : cahier de vente manuscrit, absence de suivi de stock en temps réel, aucune traçabilité fiable de qui a vendu quoi, et aucun moyen de vérifier l'activité d'une boutique sans s'y trouver physiquement. Cela pose quatre problèmes concrets que le système d'information doit résoudre :

- **Absence de visibilité en temps réel** sur le stock disponible — risque de vendre un produit déjà épuisé (survente).
- **Absence de traçabilité** — impossible de savoir avec certitude qui a encaissé quelle vente, ni de faire confiance à un historique modifiable après coup.
- **Absence de séparation des rôles** — un vendeur qui a les mêmes droits qu'un propriétaire peut, par erreur ou intentionnellement, altérer des données sensibles (prix d'achat, bénéfice, équipe).
- **Absence de vision multi-boutiques** — un propriétaire qui ne peut pas être physiquement présent dans une boutique gérée par un tiers n'a aucun moyen d'en vérifier l'activité.

0.3 Objectifs du système d'information

Intégrité et confiance

Empêcher structurellement (au niveau de la base, pas seulement de l'interface) toute survente, toute suppression d'un historique de vente, tout mouvement de stock non tracé.

Cloisonnement multi-tenant

Garantir qu'aucune donnée d'une boutique n'est jamais visible ou modifiable par une autre, sauf mécanisme explicite et volontaire (supervision, lien fournisseur).

Rôles et permissions différenciés

Modéliser précisément ce que peut faire un Propriétaire, un Vendeur, un Superviseur en lecture seule, et un administrateur de la plateforme elle-même.

Modèle économique SaaS

Adosser des fonctionnalités (supervision, fournisseurs inter-boutiques, commandes d'achat) à des paliers d'abonnement payants, activables/désactivables sans toucher au code.

DOMAINE	CONTENU
Comptes & boutiques	Inscription, rôles, rattachement d'un utilisateur à une boutique, invitations d'employés, types de boutique
Catalogue & stock	Produits, catégories, prix dégressifs par quantité, mouvements de stock, transferts entre boutiques
Ventes & clients	Caisse, panier, paiement (total ou partiel), retours, historique client, créances
Achats & fournisseurs	Fournisseurs externes, commandes d'achat, réception de stock, fournisseurs internes à la plateforme
Supervision & sécurité	Supervision en lecture seule, journal d'audit, vérification d'e-mail, révocation de session
Plateforme & abonnement	Plans d'abonnement, demandes de paiement, coordonnées de paiement de la plateforme

Règles d'élaboration d'un système d'information

1.1 La démarche méthodologique

Concevoir la base de données d'un système d'information ne consiste jamais à écrire directement des CREATE TABLE. La méthode **Merise**, largement enseignée et appliquée dans les projets francophones, impose de séparer la réflexion en **niveaux d'abstraction successifs**, chacun répondant à une question différente et volontairement indépendant du précédent :

1. Niveau conceptuel

Quoi ? Quelles informations le métier manipule-t-il, et comment sont-elles liées entre elles ? Totalement indépendant de toute technologie — le MCD serait identique si la base était MySQL, Oracle ou PostgreSQL.

2. Niveau logique

Comment structurer ? Traduction du MCD en relations (tables), avec clés primaires et clés étrangères — indépendant du SGBD précis, mais déjà structuré « à la relationnel ».

3. Niveau physique

Avec quel outil ? Implémentation réelle dans un SGBD précis — ici PostgreSQL : types de colonnes exacts, index, triggers, séquences, extensions.

POURQUOI NE PAS CONSTRUIRE DIRECTEMENT LES TABLES ?

Parce qu'une erreur de compréhension du métier, commise directement en SQL, se répercute dans tout le code applicatif qui s'appuie dessus — la corriger devient coûteux. En travaillant d'abord au niveau conceptuel (avec le métier, sans jargon technique), puis logique, puis seulement physique, chaque niveau peut être vérifié et corrigé *avant* que son coût de correction n'explose. C'est exactement l'ordre suivi par ce document : Parties 2 à 4 restent au niveau conceptuel, Partie 5 passe au niveau logique, Parties 6 et 7 au niveau physique réel de ce projet.

1.2 Règles générales d'élaboration

Au-delà de la démarche en trois niveaux, un système d'information bien construit respecte un petit nombre de règles, systématiquement appliquées dans ce projet :

- **Unicité de l'information** — une donnée n'est stockée qu'à un seul endroit. Une valeur dérivée (ex. `customers.total_spent`, le montant théorique d'une caisse) n'est tolérée que si elle est *documentée comme telle* et maintenue par un mécanisme unique et automatique (trigger) — jamais recalculée différemment à deux endroits du code.
- **Identifiant stable** — chaque entité possède une clé primaire technique, arbitraire et immuable, jamais une donnée métier qui pourrait changer (un e-mail peut être corrigé, un nom peut être renommé ; un identifiant technique, jamais).
- **Toute règle de gestion doit être rattachée à un mécanisme du modèle** — une cardinalité, une contrainte d'intégrité (CHECK, UNIQUE), ou, quand la règle porte sur plusieurs lignes ou plusieurs tables à la fois, un trigger.
- **Fidélité au métier réel** — ne modéliser que ce qui correspond à un besoin constaté, pas ce qui paraît « générique ». Ce projet a par exemple explicitement renoncé à un rôle intermédiaire « Manager » entre Propriétaire et Vendeur : dans le contexte des petites boutiques guinéennes visées, cette distinction n'a pas de réalité métier.
- **Protection de la donnée à valeur de preuve** — toute information qui constitue un historique ou une preuve (une vente, un mouvement de stock, une ligne de journal d'audit) doit être protégée contre la

1.3 Règles de gestion réelles du projet

Les règles de gestion sont les contraintes métier, exprimées en langage naturel, que le modèle de données devra ensuite traduire en cardinalités et en contraintes techniques. Voici les règles de gestion effectivement en vigueur dans ce projet — chacune sera reliée, dans les parties suivantes, au mécanisme précis qui l'applique.

#	RÈGLE DE GESTION	MÉCANISME D'APPLICATION (APERÇU)
RG1	Un utilisateur peut posséder au plus une boutique.	UNIQUE(owner_id)
RG2	Une boutique appartient à exactement un propriétaire.	owner_id NOT NULL
RG3	Une personne ne peut être Vendeur que dans une seule boutique à la fois.	Trigger prevent_multi_store_reseller
RG4	Un Super Administrateur ne peut jamais posséder de boutique.	Trigger prevent_super_admin_owning_store
RG5	Une vente, un mouvement de stock, une commande d'achat ne sont jamais supprimés physiquement.	Triggers prevent_delete / prevent_update_delete
RG6	Le stock d'un produit ne peut jamais devenir négatif.	CHECK (quantity >= 0)
RG7	Une caisse ne peut être ouverte deux fois par le même vendeur sans être fermée entre-temps.	Index unique partiel
RG8	Le prix et la quantité d'une ligne de vente déjà validée sont figés.	Trigger prevent_order_item_core_update
RG9	La supervision d'une boutique ne donne strictement aucun droit d'action.	Table séparée, uniquement des routes GET
RG10	Une commande d'achat vient soit d'un fournisseur externe, soit d'un fournisseur interne à la plateforme, jamais les deux.	Contrainte CHECK « OU exclusif »

Dictionnaire de données, associations et cardinalités

Cette partie recense, avant toute modélisation graphique, la matière première du système : quelles informations existent, et quels liens (associations) existent entre elles. C'est la première étape concrète de la démarche — le schéma visuel (MCD, Partie 4) n'en sera que la mise en forme, après nettoyage par les règles de normalisation (Partie 3).

CONVENTION COMMUNE À TOUTES LES TABLES DU DICTIONNAIRE

Pour ne pas répéter 30 fois la même information : sauf mention contraire, chaque table possède un identifiant technique `id` (entier auto-incrémenté, jamais une donnée métier), une date de création `created_at`, et les tables représentant une fiche modifiable (pas un historique) possèdent aussi `updated_at`. La quasi totalité des tables porte également un `store_id` : c'est la colonne qui réalise le **cloisonnement multi-tenant** — chaque ligne appartient à une boutique et une seule. Ces colonnes ne sont donc listées ci-dessous que lorsqu'elles portent une règle particulière.

A Comptes & boutiques

TABLE	ATTRIBUT	TYPE	RÔLE
ROLE	code	VARCHAR(20)	Référentiel fermé : SUPER_ADMIN, OWNER, SELLER
	label	VARCHAR(50)	Libellé affiché
	3 lignes fixes, jamais créées par un utilisateur		
UTILISATEUR (users)	full_name, email, phone	VARCHAR	Identité, e-mail unique (insensible à la casse)
	password_hash	VARCHAR(255)	Empreinte bcrypt, jamais le mot de passe en clair
	status	VARCHAR(20)	ACTIVE / INACTIVE / PENDING_VERIFICATION
	is_super_admin	BOOLEAN	Statut plateforme — jamais positionné par une route publique
	gender, birth_date, token_version	mixte	Profil ; compteur de révocation de session
BOUTIQUE (stores)	name, category, logo_url, phone, email, address, city, region, country	mixte	Identité et localisation
	status	VARCHAR(20)	ACTIVE / SUSPENDED / TRIAL
	supervision_code, supplier_code	VARCHAR(20)	Codes de partage uniques, non-devinables (opaques, jamais l'id numérique)
	plan_id, plan_expires_at, allow_all_sellers_void_return, receipt_settings, store_type_id	mixte	Abonnement effectif, autorisations, personnalisation du reçu
TRAVAILLE_DANS (user_store)	role_id	FK	Rôle occupé dans cette boutique précise

	permissions	JSONB	Droits fins (ex. autorisation d'annulation/retour)
	is_default_store	BOOLEAN	Boutique ouverte par défaut à la connexion
TYPE_BOUTIQUE / CATÉGORIE SUGGÉRÉE	code, label, name, display_order	mixte	Référentiel de secteurs d'activité et catégories suggérées à la création
INVITATION (store_invitations)	email, role_id, invited_by	mixte	Invitation en attente, avant que le compte existe

Associations et cardinalités identifiées

- UTILISATEUR (0,1) — POSSÈDE — (1,1) BOUTIQUE — un utilisateur possède au plus une boutique (RG1) ; une boutique a toujours exactement un propriétaire (RG2).
- UTILISATEUR (0,N) — TRAVAILLE_DANS — (1,N) BOUTIQUE, association **porteuse** (qualifiée par un rôle, des permissions) — un utilisateur peut travailler dans plusieurs boutiques dans le temps ; une boutique compte toujours au moins un membre (son propriétaire) et emploie souvent plusieurs personnes (RG3 restreint ce N,N pour le rôle Vendeur).
- BOUTIQUE (0,1) — EST_DE_TYPE — (0,N) TYPE_BOUTIQUE — un type de boutique suggère une liste de catégories de départ.
- BOUTIQUE (0,N) — ÉMET — (1,1) INVITATION, qualifiée par un rôle destiné à l'invité.

B Abonnement & plateforme

TABLE	ATTRIBUT	TYPE	RÔLE
PLAN (subscription_plans)	name, price, max_users_per_store	mixte	Offre commerciale (GRATUITE / STANDARD / PROFESSIONNEL)
	allows_supervision, allows_suppliers, allows_purchase_orders	BOOLEAN	Fonctionnalités déverrouillées par ce plan
DEMANDE_PAIEMENT (subscription_payment_requests)	payment_method, payer_phone, transaction_reference	mixte	Déclaration de paiement (Orange Money / Mobile Money / carte)
	amount_declared	NUMERIC	Calculé côté serveur, jamais saisi librement
	status, reviewed_by, reviewed_at, rejection_reason	mixte	Vérification manuelle par un Super Admin
COORDONNÉES_PAIEMENT (platform_payment_settings)	orange_money_number, mobile_money_number, paycard_info, contact_*	mixte	Table « singleton » (une seule ligne) — coordonnées réelles de la plateforme
FACTURE (invoices)	period_start/end, subscription_amount, status	mixte	Table historique, prévue pour une facturation automatisée — non branchée à ce jour (voir encadré)

Associations et cardinalités identifiées

- PLAN (1,1) — SOUSCRIT_PAR — (0,N) BOUTIQUE — chaque boutique référence un plan (par défaut GRATUITE), un plan est utilisé par de nombreuses boutiques.
- BOUTIQUE (0,N) — DEMANDE — (1,1) DEMANDE_PAIEMENT, PLAN (0,N) — CONCERNE — (1,1) DEMANDE_PAIEMENT, UTILISATEUR (0,N) — SOUMET — (1,1) DEMANDE_PAIEMENT.
- COORDONNÉES_PAIEMENT ne participe à aucune association — entité de configuration globale, volontairement isolée.

UNE TABLE LÉGUÉE, ASSUMÉE COMME TELLE

invoices a été conçue tôt dans le projet pour la facturation d'abonnement, mais l'activation des plans est restée **100% manuelle** (Super Admin), et le circuit de paiement déclaratif construit ensuite (subscription_payment_requests) a couvert le besoin réel sans jamais utiliser invoices. Elle est documentée ici par souci d'exhaustivité, mais n'est alimentée par aucun code applicatif à ce jour — un cas réel et assumé de schéma en avance sur l'usage, plutôt qu'un oubli.

C Catalogue & stock

TABLE	ATTRIBUT	TYPE	RÔLE
CATÉGORIE (categories)	name, parent_id	mixte	Hiérarchie optionnelle, propre à chaque boutique
PRODUIT (products)	name, reference, description	mixte	Fiche produit ; référence unique par boutique
	purchase_price, selling_price	NUMERIC(15,2)	Prix d'achat (jamais visible d'un tiers) et de vente
	quantity, low_stock_threshold	INTEGER	Stock courant (jamais négatif) et seuil d'alerte
	attributes	JSONB	Attributs libres selon le secteur (couleur, dimension...)
MOUVEMENT_STOCK (stock_movements)	type	VARCHAR(20)	Nature du mouvement (achat, vente, retour, ajustement, transfert, stock initial)
	quantity, unit_cost, reference_table/id	mixte	Quantité, coût, origine polymorphe du mouvement
PALIER_PRIX (product_price_tiers)	min_quantity, unit_price	mixte	Prix dégressif à partir d'une quantité (vente en gros)
TRANSFERT_STOCK (stock_transfers)	from_store_id, to_store_id, quantity, status	mixte	Transfert entre deux boutiques distinctes

Associations et cardinalités identifiées

- CATÉGORIE (0,1) — EST_SOUS_CATÉGORIE_DE — (0,N) CATÉGORIE — association **réflexive** (une catégorie peut avoir une catégorie mère).
- CATÉGORIE (0,N) — CLASSE — (0,1) PRODUIT — un produit a au plus une catégorie ; une catégorie classe plusieurs produits.
- PRODUIT (1,N) — SUBIT — (1,1) MOUVEMENT_STOCK et PRODUIT (0,N) — A — (1,1) PALIER_PRIX.
- BOUTIQUE (0,N) — ORIGINE — (1,1) TRANSFERT_STOCK et BOUTIQUE (0,N) — DESTINATION — (1,1) TRANSFERT_STOCK — deux associations **distinctes** vers la même entité BOUTIQUE (d'où CHECK from_store_id <> to_store_id).

D Ventes & clients

TABLE	ATTRIBUT	TYPE	RÔLE
CLIENT (customers)	name, phone, email, address	mixte	Fiche client, propre à chaque boutique
	total_spent, balance_due	NUMERIC	Valeurs dérivées, maintenues automatiquement
PAIEMENT_CLIENT (customer_payments)	amount, user_id	mixte	Règlement d'une créance client (paiement partiel antérieur)
COMMANDE (orders)	order_number, total_amount, amount_paid	mixte	Numéro unique par boutique, montant total et réellement encaissé
	status	VARCHAR(20)	PAID / RETURNED / PARTIALLY_RETURNED / VOIDED
	payment_method, payment_status	mixte	Espèces/Mobile Money/Carte ; payé, partiel ou en attente
	seller_id, customer_id, cash_drawer_id	FK	Qui a vendu, à qui, sur quelle caisse
LIGNE_VENTE (order_items)	quantity, unit_price	NUMERIC	Prix figé au moment de la vente
	returned_quantity	INTEGER	Seule colonne modifiable après coup (flux de retour)
CAISSE (cash_drawers)	opening/closing_balance, opening/closing_time	mixte	Session de caisse (shift) d'un vendeur
	expected_balance, status	mixte	Solde théorique (calculé), OPEN / CLOSED

Associations et cardinalités identifiées

- CLIENT (0,N) — PASSE — (0,1) COMMANDE — une vente au comptant sans fiche client est possible.
- UTILISATEUR (0,N) — VEND — (1,1) COMMANDE, BOUTIQUE (0,N) — ENREGISTRE — (1,1) COMMANDE, CAISSE (0,N) — ENCAISSE — (0,1) COMMANDE.
- COMMANDE (1,N) = LIGNE_VENTE = (0,N) PRODUIT — **association porteuse many-to-many** entre COMMANDE et PRODUIT : une commande contient toujours au moins un produit, un produit peut apparaître dans plusieurs commandes ; l'association elle-même porte la quantité et le prix.
- CLIENT (0,N) — RÈGLE — (1,1) PAIEMENT_CLIENT, UTILISATEUR (0,N) — OUVRE — (1,1) CAISSE.

E Achats & fournisseurs

TABLE	ATTRIBUT	TYPE	RÔLE
FOURNISSEUR_CONTACT (supplier_contacts)	name, phone, email, address	mixte	Carnet d'adresses libre — fournisseur hors plateforme
COMMANDE_ACHAT (purchase_orders)	reference, total_amount, status	mixte	PENDING / RECEIVED / CANCELLED
	supplier_id ou supplier_store_id	FK	Origine exclusive : fournisseur externe OU boutique-fournisseur de la plateforme
	created_by, received_by	FK	Traçabilité de la commande puis de la réception
LIGNE_ACHAT (purchase_order_items)	quantity, purchase_price	mixte	Ligne immuable après création
	supplier_product_id	FK nullable	Produit du catalogue fournisseur à l'origine de la ligne
LIEN_FOURNISSEUR (store_supplier_links)	buyer_store_id, supplier_store_id	FK	Une boutique de la plateforme devient fournisseur d'une autre
CORRESPONDANCE_PRODUIT (store_supplier_product_links)	supplier_product_id, buyer_product_id	FK	Fiche produit de l'acheteur générée pour un produit du fournisseur

Associations et cardinalités identifiées

- FOURNISSEUR_CONTACT (0,N) — FOURNIT — (0,1) COMMANDE_ACHAT **ou** BOUTIQUE (0,N) — FOURNIT_EN_INTERNE — (0,1) COMMANDE_ACHAT — deux associations mutuellement exclusives vers une même commande.
- COMMANDE_ACHAT (1,N) = LIGNE_ACHAT = (0,N) PRODUIT — même schéma d'association porteuse many-to-many que LIGNE_VENTE.
- BOUTIQUE (0,N) — EST_FOURNISSEUR_DE — (0,N) BOUTIQUE — association **réflexive** (une boutique joue tour à tour le rôle d'acheteuse et de fournisseuse selon la ligne).

F Supervision, sécurité & traçabilité

TABLE	ATTRIBUT	TYPE	RÔLE
SUPERVISION (store_supervisors)	store_id, supervisor_user_id	FK	Qui supervise quelle boutique, en lecture seule
JOURNAL_AUDIT (system_logs)	action, details	VARCHAR / JSONB	Action tracée (connexion, annulation, transfert...)
	user_id, store_id, ip_address	mixte, nullables	store_id NULL = action globale de plateforme
CODE_VÉRIFICATION (user_verification_codes)	purpose, code, expires_at, used_at	mixte	E-mail de vérification ou réinitialisation de mot de passe
NOTE (store_notes)	title, content, color, is_pinned, author_id	mixte	Bloc-note partagé de l'équipe

Associations et cardinalités identifiées

- UTILISATEUR (0,N) — SUPERVISE — (0,N) BOUTIQUE — association réflexive-like : un utilisateur peut superviser plusieurs boutiques, une boutique peut avoir plusieurs superviseurs.
- UTILISATEUR (0,N) — GÉNÈRE — (0,1) JOURNAL_AUDIT, BOUTIQUE (0,N) — CONCERNE — (0,1) JOURNAL_AUDIT — les deux cardinalités du côté JOURNAL_AUDIT sont à (0,1) car une action peut être globale (pas de boutique) ou anonymisée après suppression d'un compte.
- UTILISATEUR (0,N) — REÇOIT — (1,1) CODE_VÉRIFICATION, BOUTIQUE (0,N) — CONTIENT — (1,1) NOTE.

Règles de normalisation appliquées

Avant de dessiner le schéma conceptuel, le dictionnaire de données recensé en Partie 2 doit être **affiné** : éliminer les redondances non maîtrisées, s'assurer que chaque attribut est rattaché à la bonne entité. C'est le rôle des trois premières formes normales. Chacune est illustrée ci-dessous par un exemple réel de ce projet — un cas où la règle est strictement respectée, et un cas où une exception existe, **documentée et justifiée** plutôt qu'ignorée.

1FN Première forme normale — atomicité

Règle : chaque attribut doit contenir une valeur unique et indivisible ; aucune liste, aucun groupe répétitif dans une même colonne.

APPLIQUÉE STRICTEMENT

Un utilisateur peut travailler dans plusieurs boutiques au fil du temps. La solution non-1FN serait une colonne `users.boutiques = [12, 45]` (une liste dans une seule cellule). Ce projet extrait au contraire ce lien répétitif dans sa propre table, `user_store` — une ligne par rattachement, chaque ligne parfaitement atomique. C'est exactement le mécanisme qui, en Partie 5, transformera une association en table à part entière.

EXCEPTION DOCUMENTÉE : LES COLONNES JSONB

Les colonnes `products.attributes`, `user_store.permissions`, `stores.receipt_settings` et `system_logs.details` stockent une structure, pas une valeur atomique — un écart apparent à la 1FN. Il est assumé, pour une raison précise à chaque fois : ces informations sont **par nature variables** (les attributs pertinents d'un produit changent selon qu'il s'agit de téléphonie ou de pharmacie) et **ne sont jamais utilisées dans une jointure relationnelle** — jamais de JOIN sur une clé cachée à l'intérieur du JSON. Le type JSONB de PostgreSQL est ici un outil de modélisation délibéré, pas un contournement : il reste interrogeable (l'index GIN `idx_products_attributes`, détaillé en Partie 7, le rend même performant en recherche), ce qu'une colonne TEXT libre ne permettrait pas.

2FN Deuxième forme normale — dépendance à la clé entière

Règle : pour une table à clé composite, chaque attribut non-clé doit dépendre de *l'ensemble* de la clé, jamais d'une seule partie.

APPLIQUÉE STRICTEMENT

`product_price_tiers` a pour clé candidate naturelle le couple (`product_id`, `min_quantity`) — mais son attribut `unit_price` dépend bien des **deux** colonnes à la fois (le prix dépend du produit et du palier de quantité choisi), jamais d'une seule : aucune dépendance partielle. Le choix d'une clé primaire technique `id` plutôt que la clé composite elle-même (contrainte UNIQUE(`product_id`, `min_quantity`) conservée à côté) est un choix d'implémentation physique, sans incidence sur la conformité 2FN du modèle.

UN PIÈGE ÉVITÉ : ORDER_ITEMS

Une modélisation naïve aurait pu donner à `order_items` la clé composite (`order_id`, `product_id`). Ce projet ne le fait délibérément pas : rien n'empêche métier qu'un même produit apparaisse sur deux lignes distinctes d'une même vente (retour partiel suivi d'un rachat dans le même panier, par exemple). Une clé composite aurait

imposé à tort l'unicité du couple — la clé technique `id` reflète fidèlement que **chaque ligne de vente est un fait indépendant**, jamais fusionnable avec une autre.

3FN Troisième forme normale — dépendance directe, non transitive

Règle : un attribut non-clé ne doit dépendre que de la clé elle-même, jamais d'un autre attribut non-clé.

APPLIQUÉE STRICTEMENT

`orders` ne stocke ni le nom ni le téléphone du client, seulement `customer_id`. Les stocker directement créerait une dépendance transitive (`order_id` → `customer_id` → `customer_name`) : le nom du client dépendrait de la commande *via* le client, pas directement de la commande. En cas de correction du nom d'un client, une seule ligne change, dans une seule table — jamais de désynchronisation possible entre l'historique des ventes et la fiche client.

UNE REDONDANCE QUI N'EN EST PAS UNE : LE PRIX FIGÉ

`order_items.unit_price` pourrait sembler transitivement dépendant de `products.selling_price` *via* `product_id` — donc, en apparence, une violation de la 3FN. Ce n'est pas le cas : ce sont **deux faits distincts**, à deux instants distincts. `products.selling_price` est le prix *courant*, modifiable à tout moment ; `order_items.unit_price` est le prix *historique*, figé au moment précis de la transaction (garanti par le trigger `prevent_order_item_core_update`, Partie 7). Le même raisonnement s'applique à `purchase_order_items.purchase_price` et à `subscription_payment_requests.amount_declared`, toujours calculé au moment de la soumission et jamais recalculé ensuite, même si le prix du plan change par la suite.

DÉNORMALISATION ASSUMÉE, À DES FINS DE PERFORMANCE

`customers.total_spent` et `customers.balance_due` sont, eux, de vraies valeurs dérivées — strictement recalculables en additionnant l'historique de `orders` et `customer_payments`. Ce choix est délibéré : afficher la fiche d'un client sans recalculer une agrégation sur tout son historique à chaque ouverture d'écran. La règle de la Partie 1 (« unicité de l'information ») reste respectée dans son esprit : ces colonnes sont maintenues par un **unique point de mise à jour** dans le code applicatif (`orders.service.js` / `customers.service.js`), jamais recalculées différemment à deux endroits. À la différence de `cash_drawers.expected_balance` (Partie 7), la garantie est ici portée par la discipline du code plutôt que par un trigger — un compromis explicite, pas un oubli.

Construction du Modèle Conceptuel de Données (MCD)

Le MCD traduit en schéma graphique le dictionnaire normalisé de la Partie 3 : des **entités** (rectangles), reliées par des **associations** (nommées, parfois porteuses de leurs propres attributs), chaque lien précisant sa **cardinalité** — le nombre minimum et maximum de fois qu'une occurrence d'une entité participe à l'association.

0,1 — participe zéro ou une fois (optionnel, au plus une fois) **1,1** — participe exactement une fois (obligatoire)

0,N — participe zéro à plusieurs fois (optionnel, répétable) **1,N** — participe une à plusieurs fois (obligatoire, répétable)

POURQUOI 6 SCHÉMAS PLUTÔT QU'UN SEUL

La base compte 30 tables. Un schéma unique les réunissant toutes serait illisible pour une soutenance. Le modèle est donc découpé en **6 domaines fonctionnels**, chacun présenté avec son propre schéma — une entité qui sert de pont entre deux domaines (typiquement BOUTIQUE ou UTILISATEUR) est simplement redessinée, plus légère, dans chaque schéma où elle intervient.

A Domaine Comptes & boutiques

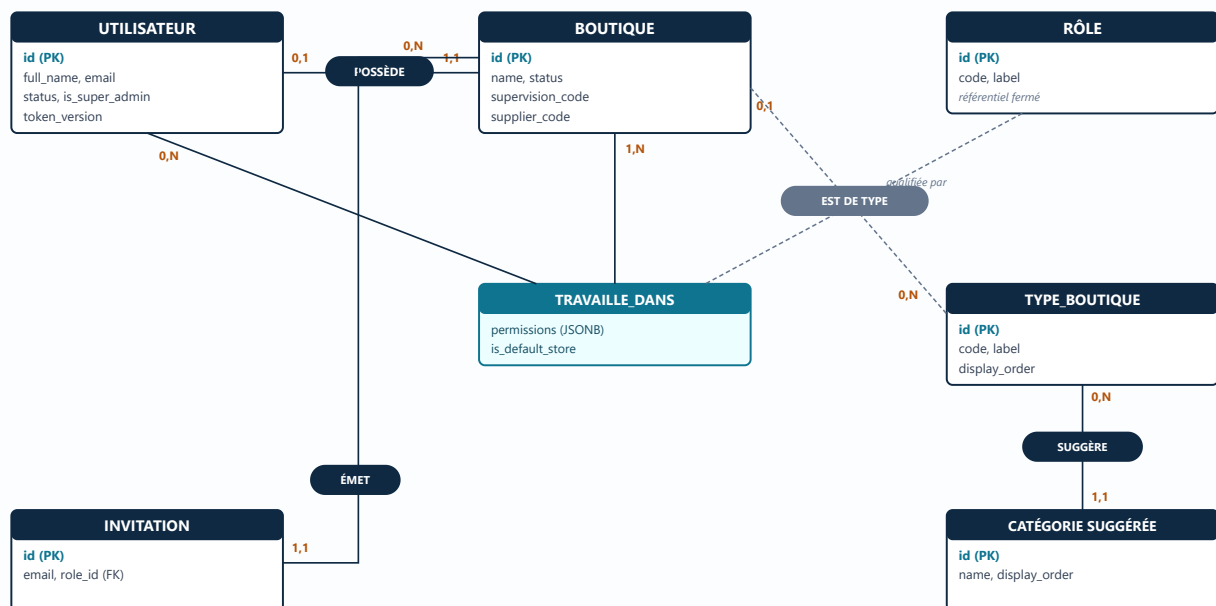


Schéma A — Comptes, boutiques et rattachements. Trait plein : association structurante. Trait pointillé : référence à une nomenclature.

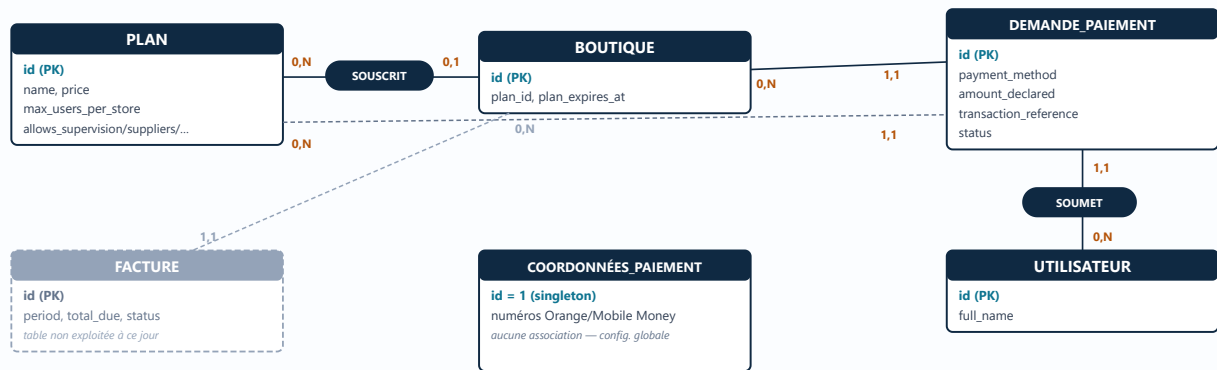


Schéma B — Abonnement, paiements et facturation. FACTURE grisée : conçue mais jamais alimentée par le code (cf. encadré Partie 2).

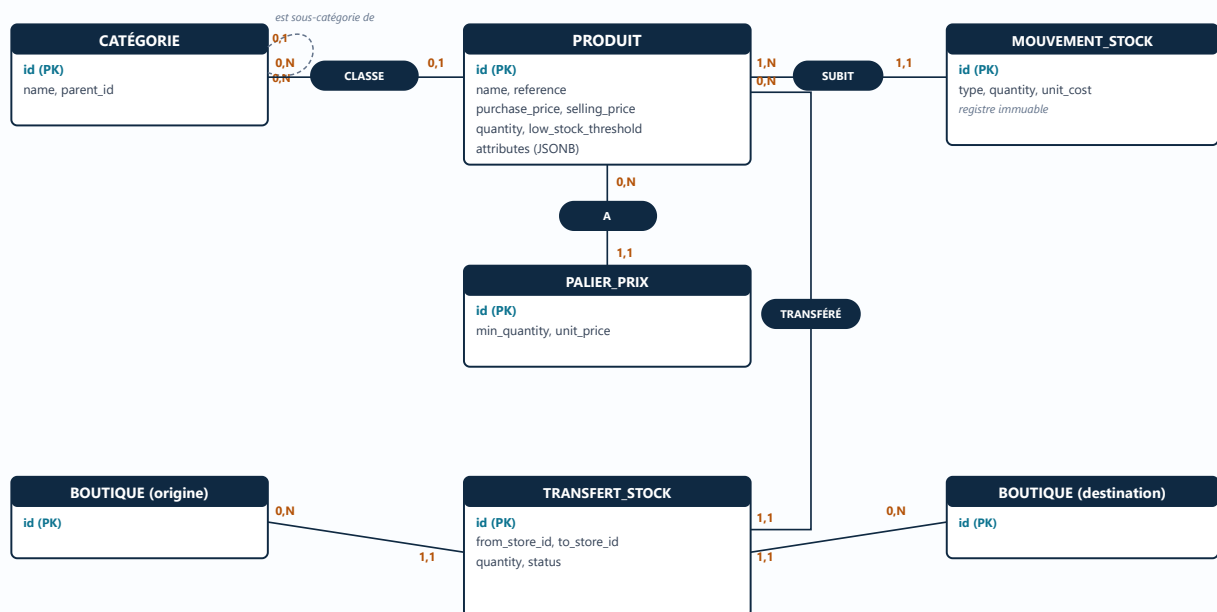


Schéma C — Catalogue, stock et transferts. TRANSFERT_STOCK relie deux fois l'entité BOUTIQUE, dans deux rôles distincts (origine / destination).

D Domaine Ventes & clients

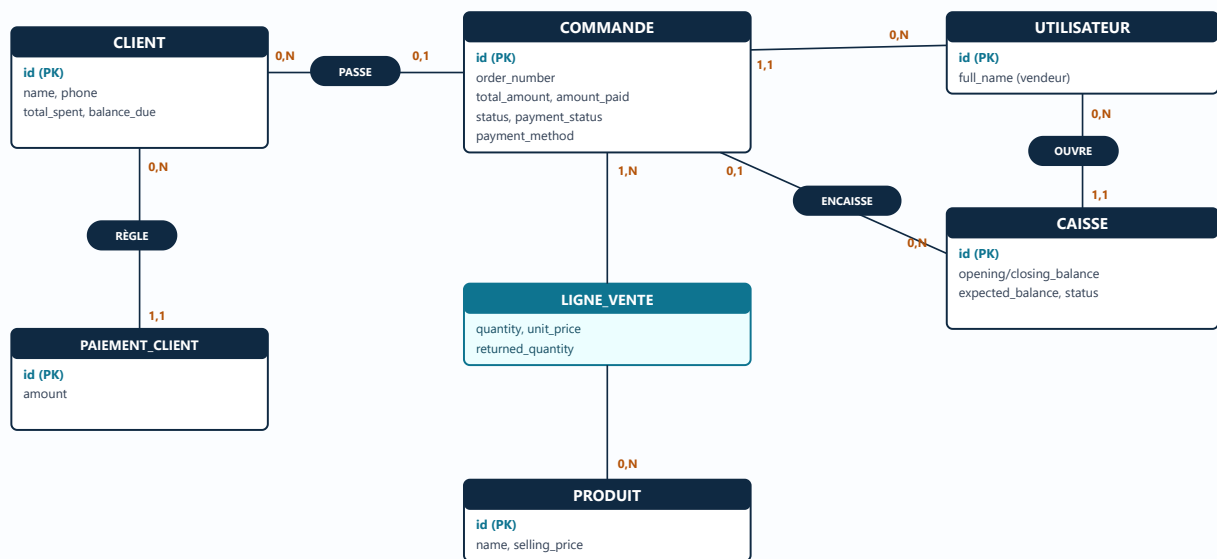


Schéma D — Ventes, caisse et clients. LIGNE_VENTE porte le prix figé et relie COMMANDE à PRODUIT (association N,N).

E Domaine Achats & fournisseurs

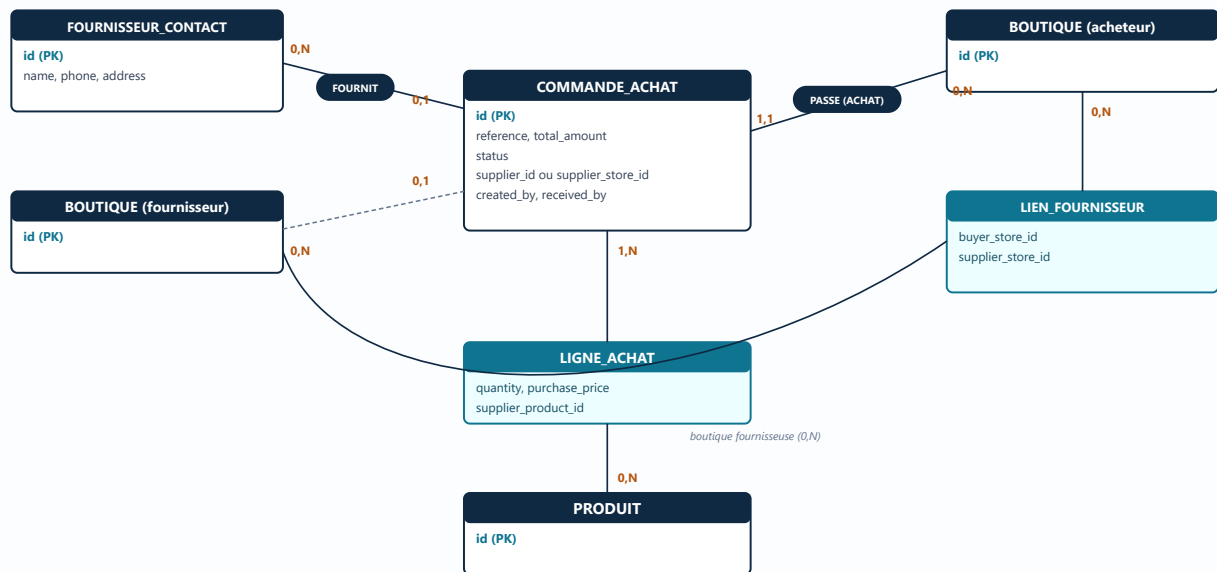


Schéma E — Achats et fournisseurs. Une COMMANDE_ACHAT vient exclusivement d'un fournisseur externe ou d'une boutique-fournisseur (traits en pointillés = branche alternative).

F Domaine Supervision, sécurité & traçabilité

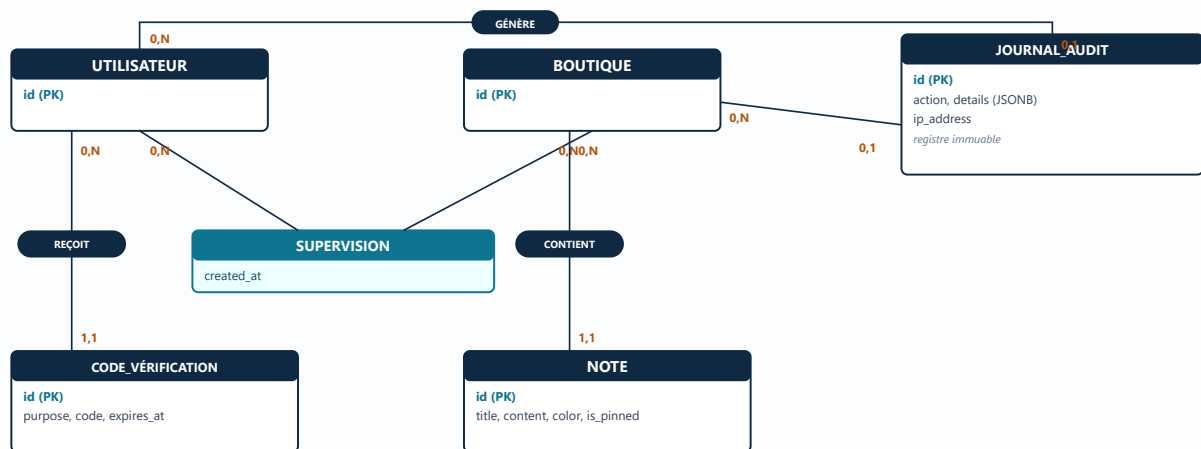


Schéma F — Supervision, journal d'audit, vérification et notes. SUPERVISION est une association N,N réflexive-like entre UTILISATEUR et BOUTIQUE.

4.7 Synthèse du MCD

DOMAINE	ENTITÉS	ASSOCIATIONS PORTEUSES (N,N)	CAS PARTICULIER NOTABLE
A — Comptes & boutiques	6	TRAVAILLE_DANS	Association 1-1 POSSÈDE (contrainte UNIQUE)
B — Abonnement & plateforme	4 + 1 hors-associations	—	COORDONNÉES_PAIEMENT : entité singleton, isolée
C — Catalogue & stock	5	—	TRANSFERT_STOCK relie deux fois BOUTIQUE (deux rôles)
D — Ventes & clients	5	LIGNE_VENTE	Prix figé, indépendant du prix courant du produit
E — Achats & fournisseurs	5	LIGNE_ACHAT, LIEN_FOURNISSEUR	Origine de COMMANDE_ACHAT strictement exclusive (OU exclusif)
F — Supervision, sécurité	4	SUPERVISION	JOURNAL_AUDIT accepte deux clés étrangères nulles à la fois

Passage au Modèle Logique de Données (MLD)

Le MCD décrit le métier ; le MLD le traduit en **relations** (tables), en appliquant un petit nombre de règles de transformation mécaniques. Aucune de ces règles n'introduit de nouvelle information — elles ne font que **reformater** ce que le MCD a déjà établi.

5.1 Les règles de transformation

CAS DU MCD	RÈGLE DE TRANSFORMATION	EXEMPLE RÉEL DE CE PROJET
Toute entité	Devient une table ; son identifiant devient la clé primaire.	PRODUIT → products(id, ...)
Association 1,1 – 1,N (père / fils)	La clé primaire du père migre comme clé étrangère dans la table fils (le côté dont le maximum de cardinalité vaut 1).	CATÉGORIE (0,N) – PRODUIT (0,1) → products.category_id
Association 1,1 stricte (les deux max valent 1)	La clé migre comme pour un 1,N, complétée d'une contrainte UNIQUE côté « propriétaire » pour interdire toute réutilisation.	UTILISATEUR (0,1) – BOUTIQUE (1,1) → stores.owner_id + UNIQUE
Association N,N (porteuse ou non)	Devient sa propre table , avec une clé étrangère vers chacune des entités participantes, et ses attributs propres le cas échéant.	COMMANDE = LIGNE_VENTE = PRODUIT → order_items(order_id, product_id, ...)
Association réflexive	La clé étrangère référence la propre table de l'entité.	CATÉGORIE (parent) → categories.parent_id → categories(id)
Association N,N réflexive	Table de liaison avec deux clés étrangères vers la même table, chacune dans un rôle distinct.	BOUTIQUE ↔ BOUTIQUE → store_supplier_links(buyer_store_id, supplier_store_id)

UN CHOIX TECHNIQUE ASSUMÉ : CLÉ TECHNIQUE PLUTÔT QUE CLÉ COMPOSITE

La règle académique classique donnerait à une table issue d'une association N,N la clé composite des deux clés étrangères (ex. (order_id, product_id)). Ce projet choisit systématiquement une **clé technique id** à la place — déjà justifié en Partie 3 (§2FN) : cela n'affaiblit aucune règle de gestion (une contrainte UNIQUE est ajoutée séparément partout où l'unicité du couple doit réellement être garantie, ex. uq_store_supervisor), et cela évite d'imposer à tort une unicité quand la réalité métier ne le demande pas (ex. order_items, Partie 3).

5.2 Notation

NOM_TABLE(clé primaire, attribut, #clé étrangère) La clé primaire est en **gras souligné** ; chaque clé étrangère est précédée de #.

5.3 Le schéma logique complet — 30 relations

Voici la traduction complète des 6 schémas conceptuels de la Partie 4, domaine par domaine.

A — Comptes & boutiques

roles(id, code, label)
users(id, full_name, email, password_hash, phone, status, is_super_admin, gender, birth_date, token_version)
stores(id, name, **#owner_id**, **#plan_id**, category, phone, email, address, city, region, country, status, supervision_code, supplier_code, plan_expires_at, **#store_type_id**, receipt_settings, allow_all_sellers_void_return)
user_store(id, **#user_id**, **#store_id**, **#role_id**, permissions, is_default_store)
store_types(id, code, label, display_order)
store_type_categories(id, **#store_type_id**, name, display_order)
store_invitations(id, **#store_id**, email, **#role_id**, **#invited_by**)

B — Abonnement & plateforme

subscription_plans(id, name, price, max_users_per_store, allows_supervision, allows_suppliers, allows_purchase_orders)
subscription_payment_requests(id, **#store_id**, **#requested_by**, **#plan_id**, payment_method, payer_phone, transaction_reference, amount_declared, status, rejection_reason, **#reviewed_by**, reviewed_at)
platform_payment_settings(id, orange_money_number, mobile_money_number, paycard_info, contact_phone, contact_whatsapp, contact_email)
invoices(id, **#store_id**, period_start, period_end, subscription_amount, transaction_fees_amount, sms_usage_amount, total_due, status)

C — Catalogue & stock

categories(id, **#store_id**, **#parent_id**, name)
products(id, **#store_id**, **#category_id**, name, reference, description, purchase_price, selling_price, quantity, low_stock_threshold, attributes, image_url, status)
stock_movements(id, **#product_id**, type, quantity, unit_cost, reference_table, reference_id, **#user_id**, note)
product_price_tiers(id, **#product_id**, min_quantity, unit_price)
stock_transfers(id, **#from_store_id**, **#to_store_id**, **#product_id**, quantity, status, **#created_by**, shipped_at, received_at)

D — Ventes & clients

customers(id, **#store_id**, name, phone, email, address, total_spent, balance_due)
customer_payments(id, **#store_id**, **#customer_id**, amount, **#user_id**)
orders(id, **#store_id**, **#customer_id**, **#seller_id**, **#cash_drawer_id**, order_number, total_amount, discount_amount, tax_amount, amount_paid, status, payment_method, payment_status)
order_items(id, **#order_id**, **#product_id**, quantity, unit_price, returned_quantity)
cash_drawers(id, **#store_id**, **#user_id**, opening_time, closing_time, opening_balance, expected_balance, closing_balance, status, note)

E — Achats & fournisseurs

supplier_contacts(id, **#store_id**, name, phone, email, address)
purchase_orders(id, **#store_id**, **#supplier_id**, **#supplier_store_id**, reference, total_amount, status, **#created_by**, **#received_by**, received_at)
purchase_order_items(id, **#purchase_id**, **#product_id**, quantity, purchase_price, **#supplier_product_id**)
store_supplier_links(id, **#buyer_store_id**, **#supplier_store_id**)
store_supplier_product_links(id, **#buyer_store_id**, **#supplier_product_id**, **#buyer_product_id**)

F — Supervision, sécurité & traçabilité

store_supervisors(id, **#store_id**, **#supervisor_user_id**)
system_logs(id, **#user_id**, **#store_id**, action, ip_address, details)
user_verification_codes(id, **#user_id**, purpose, code, expires_at, used_at)
store_notes(id, **#store_id**, **#author_id**, title, content, color, is_pinned)

Passage au Modèle Relationnel

Le MLD (Partie 5) est déjà proche du relationnel, mais reste encore un peu informel. Le **modèle relationnel**, formalisé par E. F. Codd, impose un cadre mathématique précis : une base de données est un ensemble de **relations** (tables), chaque relation un ensemble de **tuples** (lignes) sur un schéma fixe d'attributs typés, et l'intégrité de l'ensemble repose sur deux garanties : l'**intégrité d'entité** (toute ligne est identifiable de façon unique) et l'**intégrité référentielle** (toute clé étrangère renvoie soit vers une ligne qui existe réellement, soit vers rien du tout — jamais vers une ligne fantôme).

6.1 Clés candidates et clé primaire

Une **clé candidate** est tout ensemble minimal d'attributs identifiant une ligne de façon unique. Une table peut en avoir plusieurs ; une seule est choisie comme **clé primaire**. Ce projet choisit systématiquement une clé technique (Partie 1, règle « identifiant stable ») — mais plusieurs clés candidates *réelles*, issues du métier, existent et sont explicitement protégées par des contraintes UNIQUE :

TABLE	CLÉ CANDIDATE (MÉTIER)	POURQUOI ELLE N'EST PAS LA CLÉ PRIMAIRE
users	LOWER(email)	Une adresse e-mail peut, en théorie, être corrigée — une clé primaire doit rester immuable.
stores	owner_id	N'identifie pas la boutique elle-même, seulement son unicité de propriétaire (RG1).
stores	supervision_code / supplier_code	Volontairement opaques et régénérables — l'inverse d'une bonne clé primaire.
orders	(store_id, order_number)	Le numéro de vente n'a de sens qu'à l'intérieur d'une boutique — clé candidate composite.
products	(store_id, reference)	Optionnelle (un produit peut n'avoir aucune référence) — ne peut donc pas être primaire.

6.2 Intégrité référentielle : la politique de suppression

Chaque clé étrangère du modèle logique doit désormais préciser **ce qui doit arriver** si la ligne référencée est supprimée. Ce choix, invisible au niveau conceptuel, porte pourtant une vraie décision métier — il est fait ici, table par table, selon un principe simple : une donnée **dépendante** (qui n'a aucun sens sans son parent) suit son parent ; une donnée qui garde une **valeur propre** (historique, document) doit lui survivre, quitte à perdre le lien.

POLITIQUE	SIGNIFICATION	EXEMPLES RÉELS
CASCADE	Supprime aussi les lignes dépendantes	Boutique supprimée → ses produits, catégories, clients, notes disparaissent avec elle
RESTRICT	Interdit la suppression tant que des dépendances existent	Un produit déjà vendu ne peut pas être supprimé (order_items.product_id) ; une boutique-fournisseur ne peut pas être supprimée si des transferts existent
SET NULL	Rompt le lien, mais conserve la ligne	Auteur d'une note qui quitte la boutique → la note reste, author_id devient NULL

Sur `orders` : `store_id` est en CASCADE (une vente n'existe pas indépendamment de sa boutique), `seller_id` est en RESTRICT (un compte vendeur ayant déjà vendu ne peut jamais être supprimé — l'historique de vente prime), et `customer_id` est en SET NULL (une fiche client peut être supprimée sans perdre l'historique des ventes qui lui étaient liées). Trois clés étrangères de la même table, trois politiques différentes, chacune reflétant une règle de gestion distincte.

6.3 Les opérations relationnelles à l'œuvre

Le modèle relationnel n'est pas qu'une structure statique : il définit aussi les opérations qui permettent d'en extraire de l'information — sélection (filtrer des lignes), projection (choisir des colonnes) et surtout **jointure** (recomposer, à la lecture, ce que la normalisation a volontairement séparé à l'écriture). Une requête réelle de l'application illustre les trois à la fois :

```
SELECT o.id, o.order_number, o.total_amount, c.name AS customer_name, u.full_name AS seller_name
FROM orders o
LEFT JOIN customers c ON c.id = o.customer_id -- projection + jointure : recompose le nom du client
JOIN users u ON u.id = o.seller_id -- jointure : recompose le nom du vendeur
WHERE o.store_id = $1 AND o.status != 'VOIDED' -- sélection : filtre boutique + statut
```

Aucune de ces trois informations (client, vendeur, montant) n'est dupliquée nulle part dans la base — c'est exactement le résultat recherché en Partie 3 (3FN). Le modèle relationnel garantit que cette reconstitution, bien que faisant intervenir trois tables séparées, reste à la fois **correcte** (l'intégrité référentielle empêche une jointure de « fuir » vers une ligne inexistante) et **performante** — à condition d'indexer les bonnes colonnes, sujet de la Partie 7.

Création de la base PostgreSQL

Dernière étape : traduire le modèle relationnel en **PostgreSQL réel**. Cette partie n'est plus générique — chaque concept est immédiatement relié à la table, la colonne ou le mécanisme exact qui l'utilise dans ce projet. La base est intégralement définie par les scripts SQL de backend/database/ (32 fichiers, appliqués dans l'ordre par run-migrations.js), seule source de vérité du schéma — jamais un ORM.

7.1 Types de données PostgreSQL utilisés

TYPE	USAGE DANS CE PROJET	POURQUOI CE TYPE PRÉCISÉMENT
SERIAL	Toutes les clés primaires techniques	Entier auto-incrémenté — crée automatiquement une séquence (Partie 7.5)
NUMERIC(15,2)	Tous les montants (prix, totaux, soldes)	Précision exacte en base 10 — jamais FLOAT/REAL, dont l'imprécision binaire est inacceptable pour de l'argent
TIMESTAMPTZ	Toutes les dates d'horodatage	Stocke un instant absolu (converti en UTC), jamais une heure « locale » ambiguë
JSONB	attributes, permissions, details, receipt_settings	Structure binaire indexable (Partie 3) — contrairement à JSON (texte brut, non indexable)
INET	system_logs.ip_address	Type réseau natif — valide le format à l'écriture, permet des opérateurs réseau (<<) si besoin un jour
VARCHAR(n)	Champs à longueur métier bornée (codes, statuts, noms courts)	La limite documente une contrainte réelle (ex. code VARCHAR(6) pour un code à 6 chiffres)
TEXT	Contenu libre et long (adresses, descriptions, notes)	Aucune limite arbitraire là où le métier n'en impose pas

UN PIÈGE RÉEL DU DRIVER « PG » AVEC NUMERIC ET BIGINT

PostgreSQL garantit la précision exacte de NUMERIC, mais le driver Node.js pg renvoie par défaut ces valeurs sous forme de **chaînes de caractères** (pour ne jamais risquer de perte de précision côté JavaScript, qui ne connaît qu'un seul type numérique flottant). Sans correction, ce projet a réellement rencontré `item.unitPrice.toFixed` `is not a function` et des comparaisons de totaux faussées (`"9" >= "10"` vaut `true` en comparaison de chaînes). Corrigé une seule fois, à la source, dans `backend/src/config/db.js` :

`types.setTypeParser(types.builtins.NUMERIC, parseFloat)` et l'équivalent pour INT8 (le résultat d'un `SUM()` sur une colonne entière). Un exemple concret de concept PostgreSQL (précision du type) qui exige une décision délibérée côté application.

7.2 Contraintes d'intégrité

PostgreSQL offre cinq familles de contraintes ; ce projet les utilise toutes, chacune pour un rôle précis et jamais interchangeable :

PRIMARY KEY

Identité de chaque ligne — implique NOT NULL + UNIQUE, indexée automatiquement. 30 occurrences, une par table.

FOREIGN KEY

Intégrité référentielle (Partie 6.2) — assortie d'une politique ON DELETE explicite à chaque fois, jamais laissée au comportement par défaut.

UNIQUE

Une clé candidate réelle (Partie 6.1) — simple (`users.email`) ou composite (`orders(store_id, order_number)`).

CHECK

Une règle de gestion vérifiable **sur une seule ligne** — au-delà, il faut un trigger (Partie 7.4).

Deux usages de CHECK qui vont au-delà de la simple validation de valeur

```
-- OU EXCLUSIF : une commande d'achat vient d'une seule origine, jamais des deux
ALTER TABLE purchase_orders ADD CONSTRAINT chk_purchase_orders_supplier_xor
CHECK ((supplier_id IS NOT NULL) <> (supplier_store_id IS NOT NULL));

-- SINGLETON : une seule ligne de configuration plateforme peut jamais exister
CREATE TABLE platform_payment_settings (
  id SMALLINT PRIMARY KEY DEFAULT 1 CHECK (id = 1),
  ...
);
```

Le second exemple est particulièrement instructif : en interdisant toute valeur de `id` autre que 1, la contrainte CHECK transforme une contrainte de *valeur* en contrainte de *cardinalité de table* — au plus une ligne pourra jamais exister, garanti par PostgreSQL lui-même, jamais par une discipline de code.

7.3 Les index

Un index accélère la lecture au prix d'un coût d'écriture et d'espace disque. PostgreSQL propose plusieurs structures d'index ; ce projet en utilise quatre familles distinctes, chacune choisie pour un problème précis.

B-Tree — l'index par défaut

Structure arborescente équilibrée, adaptée aux comparaisons (=, <, >, tri, BETWEEN). Une PRIMARY KEY ou une contrainte UNIQUE crée *toujours* un index B-Tree implicite ; ce projet en ajoute par ailleurs près de 40 explicitement, chaque fois pour accélérer une requête réellement exécutée par l'application — jamais « au cas où ».

```
-- Le tableau de bord filtre systématiquement par boutique, vendeur et date : l'index composite
-- suit exactement l'ordre des colonnes du WHERE pour être utilisable par le planificateur.
CREATE INDEX idx_orders_store_seller_date ON orders (store_id, seller_id, created_at DESC);
```

Index unique partiel — une contrainte conditionnelle

PostgreSQL permet d'ajouter une clause WHERE à un index. Combinée à UNIQUE, elle exprime une règle de gestion qu'aucune contrainte CHECK classique ne peut formuler : « unique, mais seulement parmi les lignes qui vérifient telle condition ». Trois règles de gestion entières du projet tiennent dans une seule ligne SQL grâce à cette fonctionnalité :

```
-- RG7 : une seule caisse OUVERTE à la fois par vendeur (des caisses fermées peuvent s'accumuler)
CREATE UNIQUE INDEX uq_cash_drawers_one_open_per_user
ON cash_drawers (user_id, store_id) WHERE status = 'OPEN';

-- Une seule boutique par défaut par utilisateur (les autres rattachements restent non-défaut)
CREATE UNIQUE INDEX uq_user_store_one_default
ON user_store (user_id) WHERE is_default_store = TRUE;

-- Anti-spam : une seule demande de paiement EN ATTENTE par boutique à la fois
CREATE UNIQUE INDEX uq_payment_requests_one_pending_per_store
ON subscription_payment_requests (store_id) WHERE status = 'PENDING';
```

Sans la clause WHERE, la première règle interdirait à un vendeur d'avoir *jamais* deux caisses dans sa carrière — bien plus strict que ce que le métier demande (RG7 : jamais deux caisses **ouvertes en même temps**).

GIN — recherche dans du texte approximatif et du JSONB

L'index GIN (*Generalized Inverted Index*) indexe, pour une valeur composite, chacun de ses **composants** plutôt que la valeur entière — indispensable dès qu'on cherche « à l'intérieur » d'une colonne.

```
-- Recherche produit tolérante aux fautes de frappe (extension pg_trgm, Partie 7.6)
CREATE INDEX idx_products_name_trgm ON products USING GIN (name gin_trgm_ops);

-- Interroger l'intérieur d'un attribut JSONB sans scanner toute la table
CREATE INDEX idx_products_attributes ON products USING GIN (attributes);
```

Le premier rend une recherche du type "tel phon" ≈ "téléphone" réellement performante à grande échelle (index sur les trigrammes — groupes de 3 caractères — plutôt qu'un LIKE '%...%' qui ignore tout index). Le second permet à products.attributes (JSONB, Partie 3.1) de rester **interrogeable** malgré sa structure variable — la contrepartie technique qui rend cette exception à la 1FN acceptable.

RECENSEMENT COMPLET

Le schéma compte, au total, $\approx$ **55 index** : 30 index de clé primaire (implicites), une douzaine de contraintes **UNIQUE** (métier, simples ou composites), 5 index uniques **partiels**, 2 index **GIN**, et le reste en index B-Tree de performance — presque toujours composites et commençant par `store_id`, reflet direct du cloisonnement multi-tenant (Partie 2) : la quasi-totalité des requêtes de l'application filtrent d'abord sur la boutique courante.

7.4 Les triggers

Un **trigger** (déclencheur) exécute une fonction écrite en **PL/pgSQL** (le langage procédural natif de PostgreSQL) automatiquement, à chaque INSERT, UPDATE ou DELETE sur une table — BEFORE pour intercepter et éventuellement bloquer l'opération, AFTER pour réagir une fois qu'elle a eu lieu. C'est l'outil qui prend le relais exactement là où CHECK s'arrête (Partie 7.2) : une règle qui doit consulter **une autre ligne, une autre table, ou l'ancienne valeur** de la ligne modifiée.

POURQUOI APPLIQUER DES RÈGLES MÉTIER DANS LA BASE PLUTÔT QUE DANS LE CODE ?

Défense en profondeur (même principe que l'anti-survente sur `products.quantity`, déjà mentionné en Partie 2) : le code applicatif vérifie déjà ces règles avant d'écrire, mais un trigger garantit qu'elles tiennent **même face à un bug futur, un accès direct à la base, ou une seconde application qui écrirait dans les mêmes tables sans connaître les règles du code Node.js actuel**. Le trigger est la dernière ligne de défense, jamais la seule.

Trois fonctions génériques, réutilisées telles quelles

FONCTION	ÉVÉNEMENT	COMPOTEMENT	TABLES QUI L'UTILISENT
<code>set_updated_at()</code>	BEFORE UPDATE	Force <code>NEW.updated_at = NOW()</code> — impossible à oublier ou à falsifier depuis le code applicatif	<code>users</code> , <code>stores</code> , <code>products</code> , <code>customers</code> , <code>store_notes</code>
<code>prevent_delete()</code>	BEFORE DELETE	RAISE EXCEPTION systématique — la ligne a une valeur d'historique (RG5)	<code>orders</code> , <code>order_items</code> , <code>purchase_orders</code> , <code>stock_transfers</code>
<code>prevent_update_delete()</code>	BEFORE UPDATE OR DELETE	Registre totale ment immuable — ni modification, ni suppression	<code>stock_movements</code> , <code>system_logs</code> , <code>purchase_order_items</code>

Sept fonctions spécifiques, une règle de gestion chacune

FONCTION	ÉVÉNEMENT	TABLE	RÈGLE DE GESTION APPLIQUÉE
<code>prevent_order_item_core_update()</code>	BEFORE UPDATE	<code>order_items</code>	RG8 — quantité/prix/produit figés après création ; seule <code>returned_quantity</code> reste modifiable
<code>update_cash_drawer_expected_balance()</code>	AFTER INSERT	<code>orders</code>	Additionne <code>amount_paid</code> au solde théorique de la caisse ouverte, si le paiement est en espèces
<code>prevent_multi_store_reseller()</code>	BEFORE INSERT/UPDATE	<code>user_store</code>	RG3 — un Vendeur ne peut être rattaché qu'à une seule boutique à la fois
<code>prevent_conflicting_invitation()</code>	BEFORE INSERT/UPDATE	<code>store_invitations</code>	Même règle RG3, appliquée en amont, avant même que le compte n'existe
<code>prevent_super_admin_owning_store()</code>	BEFORE INSERT/UPDATE OF <code>owner_id</code>	<code>stores</code>	RG4 — un Super Admin ne peut jamais devenir propriétaire d'une boutique
<code>prevent_promoting_store_owner_to_admin()</code>	BEFORE UPDATE OF <code>is_super_admin</code>	<code>users</code>	Symétrique de RG4 — impossible de promouvoir un propriétaire en exercice

strip_store_access_on_admin_promotion()	AFTER UPDATE OF is_super_admin	users	Une fois promu (donc garanti sans boutique), tous les rattachements restants sont retirés
---	--------------------------------------	-------	---

LIRE UN TRIGGER RÉEL

prevent_order_item_core_update illustre l'accès aux deux versions de la ligne, seule capacité qu'un trigger BEFORE UPDATE offre et qu'un CHECK ne peut pas reproduire :

```
CREATE OR REPLACE FUNCTION prevent_order_item_core_update()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.quantity <> OLD.quantity OR NEW.unit_price <> OLD.unit_price OR NEW.product_id <> OLD.product_id THEN
        RAISE EXCEPTION 'Une ligne de vente validée ne peut pas être modifiée..';
    END IF;
    RETURN NEW; -- laisse passer les autres colonnes (ex. returned_quantity)
END;
$$ LANGUAGE plpgsql;
```

7.5 Les séquences

Une colonne SERIAL n'est pas un type au sens strict : c'est un raccourci PostgreSQL qui crée, en une seule instruction, **trois objets** — une colonne INTEGER, un objet **séquence** indépendant nommé automatiquement (table_colonne_seq), et une valeur par défaut DEFAULT nextval('table_colonne_seq'). La séquence est ensuite « possédée » par la colonne (OWNED BY) : elle est supprimée automatiquement si la colonne l'est.

```
-- Ce que products.id = SERIAL PRIMARY KEY crée réellement :  
CREATE SEQUENCE products_id_seq;  
CREATE TABLE products (  
  id INTEGER NOT NULL DEFAULT nextval('products_id_seq') PRIMARY KEY,  
  ...  
);  
ALTER SEQUENCE products_id_seq OWNED BY products.id;
```

Les 30 tables de ce schéma utilisent SERIAL, à une exception délibérée près : platform_payment_settings.id est un simple SMALLINT DEFAULT 1 — une séquence n'aurait aucun sens ici, la contrainte CHECK (id = 1) (Partie 7.2) interdisant de toute façon qu'une deuxième ligne existe jamais. Le schéma compte donc **29 séquences**, chacune allouant des identifiants strictement croissants, uniques, et jamais réutilisés — y compris après une suppression.

UN COMPORTEMENT NORMAL, SOUVENT PERÇU À TORT COMME UN BUG

Une séquence PostgreSQL **n'est pas transactionnelle** : un appel à nextval() consomme une valeur même si la transaction qui l'a demandée est ensuite annulée (ROLLBACK). Des « trous » dans une suite d'identifiants (ex. un produit id=41 suivi directement de id=43) sont donc parfaitement normaux et sans conséquence — la clé primaire garantit l'unicité, jamais la continuité.

7.6 Extensions PostgreSQL activées

EXTENSION	FONCTION APPORTÉE	USAGE EXACT DANS CE PROJET
pgcrypto	gen_random_uuid()	Génère les codes de partage non-devinables (stores.supervision_code, supplier_code) — un UUID aléatoire tronqué, jamais l'id numérique de la boutique (Partie 2, domaine A)
pg_trgm	Similarité de chaînes par trigrammes	Alimente idx_products_name_trgm (Partie 7.3) — recherche produit tolérante aux fautes de frappe

7.7 Vue d'ensemble du schéma physique

30

TABLES

≈55

INDEX

10

FONCTIONS
TRIGGER

29

SÉQUENCES

2

EXTENSIONS

Synthèse

Ce document a suivi, du début à la fin, le chemin qui sépare une idée métier d'une base de données en production : la **problématique** (Partie 0) a d'abord été traduite en **règles de gestion** vérifiables (Partie 1), recensées dans un **dictionnaire de données** (Partie 2), affinées par les **règles de normalisation** (Partie 3), formalisées en un **schéma conceptuel** découpé par domaine (Partie 4), traduites en **relations logiques** (Partie 5) puis **relationnelles** strictes (Partie 6), et enfin implémentées avec les **mécanismes réels de PostgreSQL** — types, contraintes, index, triggers, séquences (Partie 7).

8.1 Les objectifs de la Partie 0, revisités

OBJECTIF FIXÉ (PARTIE 0)	MÉCANISME(S) QUI LE CONCRÉTISE
Intégrité et confiance	CHECK (<code>quantity >= 0</code>), triggers <code>prevent_delete/prevent_update_delete</code> sur tout historique de preuve
Cloisonnement multi-tenant	<code>store_id</code> sur la quasi-totalité des tables, systématiquement en tête des index composites
Rôles et permissions différenciés	Référentiel fermé <code>roles</code> , association porteuse <code>user_store</code> , table séparée et strictement GET pour la supervision
Modèle économique SaaS	<code>subscription_plans</code> et ses booléens de fonctionnalité, vérifiés à chaque requête (jamais mis en cache dans le jeton)

8.2 Ce que ce schéma illustre

Au-delà du cas particulier de cette plateforme, la base de données construite ici illustre des décisions de modélisation transférables à tout système d'information : le choix systématique d'une clé technique plutôt qu'une clé composite ou métier ; la distinction rigoureuse entre une redondance *non maîtrisée* (à proscrire) et une dénormalisation *documentée* (à assumer, pour la performance ou pour capturer un fait historique) ; l'usage des triggers comme **dernier rempart** d'une règle de gestion, jamais comme son unique gardien ; et l'exploitation des types et index avancés de PostgreSQL (JSONB, GIN, index partiels) non comme une curiosité technique, mais comme la réponse directe à un besoin métier précis, à chaque fois nommé et justifié.

EN UNE PHRASE

Chaque table, chaque contrainte et chaque index de cette base répond à une question métier posée en Partie 0 — rien n'a été ajouté « parce que c'est la bonne pratique » sans qu'un besoin réel, tracé dans ce document, ne le justifie.